Here are the 2-mark answers and explanations for Internal Test 2 (Part A):

Question 1:

> **Apply the concept of polymorphism and state the output.**

```
class Device {
public: virtual void start() { cout<<"Device"; }
};
class Sensor : public Device {
public: void start() override { cout<<"Sensor"; }
};
int main() {
    Device* d = new Sensor();
    d->start();
}
```

(CO2, Apply, 2 Marks)

- **Output:** `Sensor`
- **Justification:** Because `start()` is declared as `virtual` in the base class `Device` and overridden in the derived class `Sensor`, runtime polymorphism is triggered. Even though `d` is a `Device*` pointer, it points to a `Sensor` object in heap memory, so `Sensor::start()` is called dynamically at runtime.

Question 2:

> **If a derived class constructor does not explicitly call the base class constructor, which constructor will be invoked automatically? Give a short justification.** *(CO2, Understand, 2 Marks)*

- **Answer:** The **default (parameterless) constructor** of the base class.
- **Justification:** When a derived class object is created, its base class part must be initialized first. If you don't explicitly specify a base constructor in the derived class's member initializer list, the compiler automatically attempts to call the base class's default constructor. (If the base class only has a parameterized constructor and no default constructor, it will result in a compilation error).

Question 3:

> **Differentiate full from partial template specialization.** *(CO3, Apply, 2 Marks)*

- **Answer:**
  - **Full Template Specialization:** You provide a completely customized implementation of a template for a **specific, single data type** (e.g., specializing `template <typename T> class MyClass` entirely for `int`). The template parameter list becomes empty (`template <>`).
  - **Partial Template Specialization:** You specialize a template for a **subset of types** or specific combinations (e.g., when a class has two template parameters `T1` and `T2`, you specialize it only for the case where `T2` is a `int`, leaving `T1` generic).

Question 4:

> **Which statement is true about the above code? Justify your answer.** *A) The compiler generates a default constructor for Base since it has a parameterized one. B) Derived must explicitly call Base (a) in its initializer list to compile successfully. C) The y (b) initialization happens before Base (a). (CO2, Apply, 2 Marks)*

- **Correct Answer: B) Derived must explicitly call Base (a) in its initializer list to compile successfully.** *(Note: Option C is false because base constructors always execute before derived members, regardless of the order in the initializer list).*
- **Justification:** Because the `Base` class defines a parameterized constructor (`Base(int a)`), the compiler **does not** generate a default constructor. Therefore, the derived class `Derived` **must** explicitly invoke the base constructor (`Base(a)`) in its initializer list; otherwise, the code fails to compile.

Here are the solutions for the remaining Part A questions (Q5 and Q6.a.i & Q6.a.ii) from the test paper:

Question 5:

> **Why is this code valid? Justify your answer.**
>
> ```
> class Base {
> public: virtual Base* create() { return new Base(); }
> };
> class Derived : public Base {
> public: Derived* create() override { return new Derived(); }
> };
> ```
>
> *A) Because override ignores return type mismatch.*
> *B) Because `Derived*` is a covariant return type of `Base*`.*
> *C) Because `create()` in Derived hides the base version.*
> *D) It is not valid and will cause an error.*
> *(CO2, Apply, 2 Marks)*

- **Correct Answer: B) Because `Derived*` is a covariant return type of `Base*`.**
- **Justification:** In C++, when overriding a virtual function, a derived class can return a pointer/reference to a *derived type* (`Derived*`) instead of the base type (`Base*`) declared in the parent class. This feature is known as **covariance**, making the code completely valid.

Question 6.a.i:

> **What is the output? Justify your answer.**
>
> ```
> class Base {
> public: virtual ~Base() { cout << "Base destroyed "; }
> ```

```
};
class Derived : public Base {
public: ~Derived() { cout << "Derived destroyed "; }
};
int main() {
    Base* b = new Derived();
    delete b;
}
```

A) Base destroyed
B) Derived destroyed Base destroyed
C) Base destroyed Derived destroyed
D) Undefined behavior
(CO2, Understand, 2 Marks)

- **Correct Answer: B) Derived destroyed Base destroyed**
- **Justification:** Because the base class destructor is marked `virtual`, deleting a derived object via a base class pointer (`delete b`) ensures polymorphism works correctly for destructors. It first executes the **Derived destructor**, and then automatically chains up to execute the **Base destructor**.

Question (Following 6.a.ii):

What happens if `cout<<b.y;` is uncommented? Justify your answer. *A) Prints 1 2*
B) Prints 1 then compile error
C) Prints garbage value for y
D) Runs fine, printing 1 2 because of inheritance
(CO2, Apply, 2 Marks)

- **Correct Answer: B) Prints 1 then compile error** (or simply: **Compile Error**)
- **Justification:** Because of **object slicing** (`Base b = d;`), the object `b` is sliced and only contains the `Base` part. It has no knowledge of `y` (which belongs to `Derived`), so trying to access `b.y` results in a compilation error.

Question 6.a.iii:

Explain the concept of function template specialization with an example program.
(CO3, Apply, 4 Marks)

- **Explanation:** Function template specialization allows you to provide a specific implementation of a function template for a particular data type when the generic template logic isn't suitable.
- **Example Program:**

```
#include <iostream>
using namespace std;

// Generic Template
template <typename T>
```

```
void printData(T val) {
    cout << "Generic: " << val << endl;
}

// Full Template Specialization for 'char'
template <>
void printData<char>(char val) {
    cout << "Specialized Character: " << val << endl;
}

int main() {
    printData(100);     // Calls generic template
    printData('Z');     // Calls specialized template for char
    return 0;
}
```

Question 6.b.i:

What will the output be? Justify your answer.

```
class Base { virtual void fun() {} };
class Derived : public Base {};
int main() {
    Base* b = new Base();
    Derived* d = dynamic_cast<Derived*>(b);
    if (d) cout << "Cast successful";
    else cout << "Cast failed";
}
```

A) Cast successful
B) Cast failed
C) Compile error because dynamic_cast is invalid here
D) Runtime crash
(CO2, Understand, 2 Marks)

- **Correct Answer: B) Cast failed**
- **Justification:** `dynamic_cast` performs safe downcasting at runtime. Here, pointer `b` is pointing to a plain `Base` object, *not* a `Derived` object. Therefore, the downcast is invalid, `dynamic_cast` returns `nullptr`, and it prints `"Cast failed"` without crashing.

Question 6.b.ii:

What will be the exact output when `C obj;` is created? Justify your answer. *A) CBA*
B) ABC
C) BAC
(CO2, Apply, 2 Marks)

- **Correct Answer: B) ABC**
- **Justification:** When a derived object (`C`) is created, constructors execute in order of inheritance from **base to derived**. Therefore, `A`'s constructor runs first, then `B`'s, and finally `C`'s, printing `"A B C "`.

Send over the next part whenever you're ready!